作文自动评分系统设计文档

1. 文档目标

本文档基于当前代码实现，描述作文自动评分系统的软件架构与关键设计，覆盖：

- 分解视图（模块分解图）
- 执行视图（执行流程图）
- 实现视图（源码结构、主要类/函数）
- 部署视图
- 动态建模图形

项目目录：`.\aeg`

2. 系统概述

系统是一个本地运行的自动评分原型，采用前后端分离但同机部署的轻量方案：

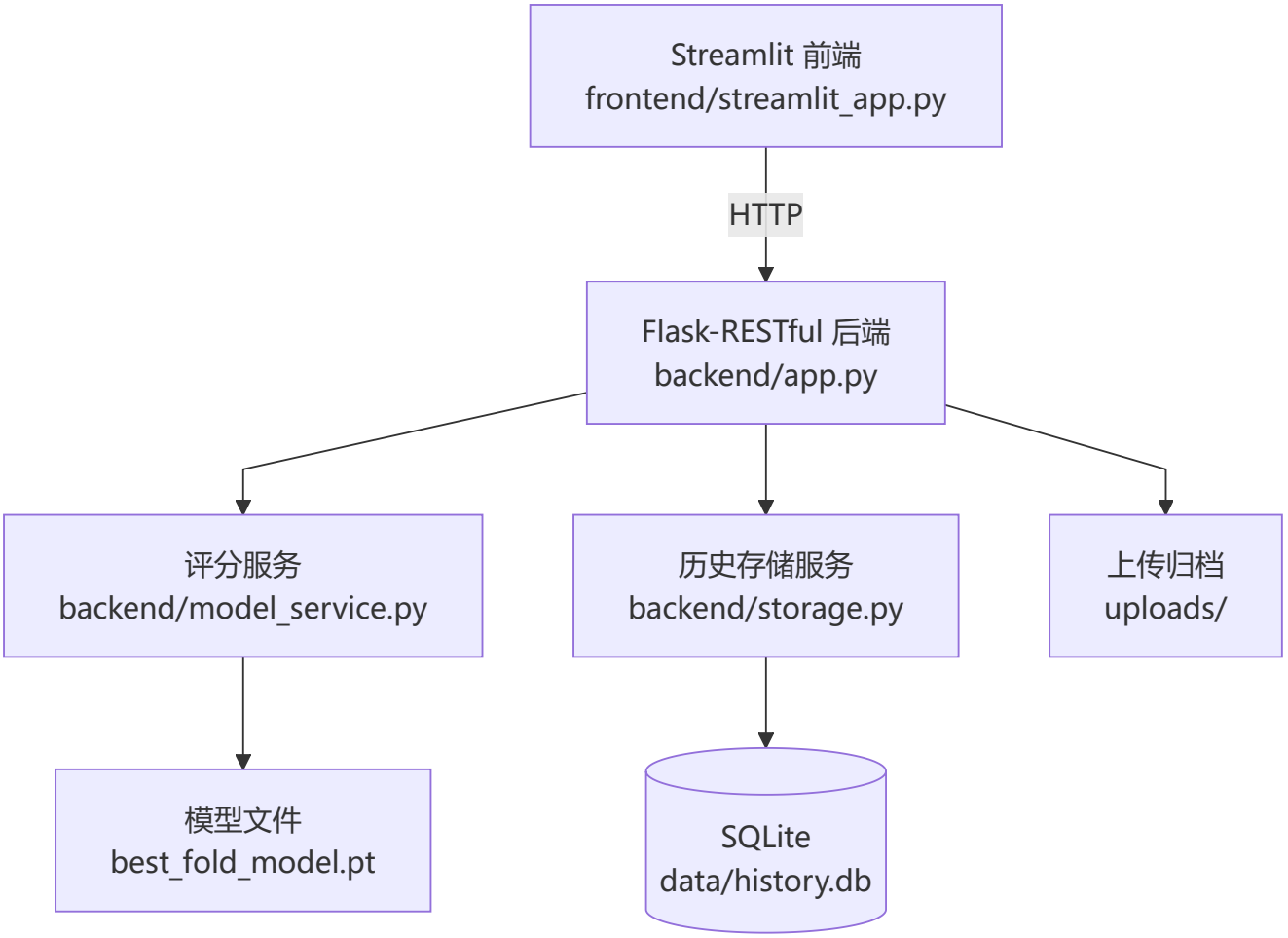
- 前端使用 `Streamlit` 提供交互页面
- 后端使用 `Flask-RESTful` 提供 HTTP API
- 模型推理使用 `PyTorch + transformers`
- 历史数据持久化使用 `SQLite`
- 上传文件归档到本地目录

系统主要完成四类任务：

- 单篇作文评分
- 批量作文评分
- 文本分析与高频词提取
- 历史记录管理（查询、详情、删除、导出）

3. 架构设计

3.1 总体架构



3.2 设计原则

- 分层明确：界面、接口、推理、存储分离
- 职责单一：模型相关逻辑只在 `EssayScoringService` 中实现
- 易于演示：所有组件支持本地直接启动
- 易于追溯：每次评分保留结构化历史和原始归档

4. 分解视图（模块分解图）

4.1 模块职责说明

前端模块

- 单篇评分页
  - 负责作文题目选择、文本输入、结果展示
- 批量评分页
  - 负责 CSV 上传、内容预览、逐篇结果展示
- 历史记录页

    - 负责筛选、分页、详情查看、删除、导出

后端接口模块

- 接收 HTTP 请求
- 参数校验
- 调用评分服务
- 调用文本分析
- 调用存储服务
- 统一返回 JSON

模型推理模块

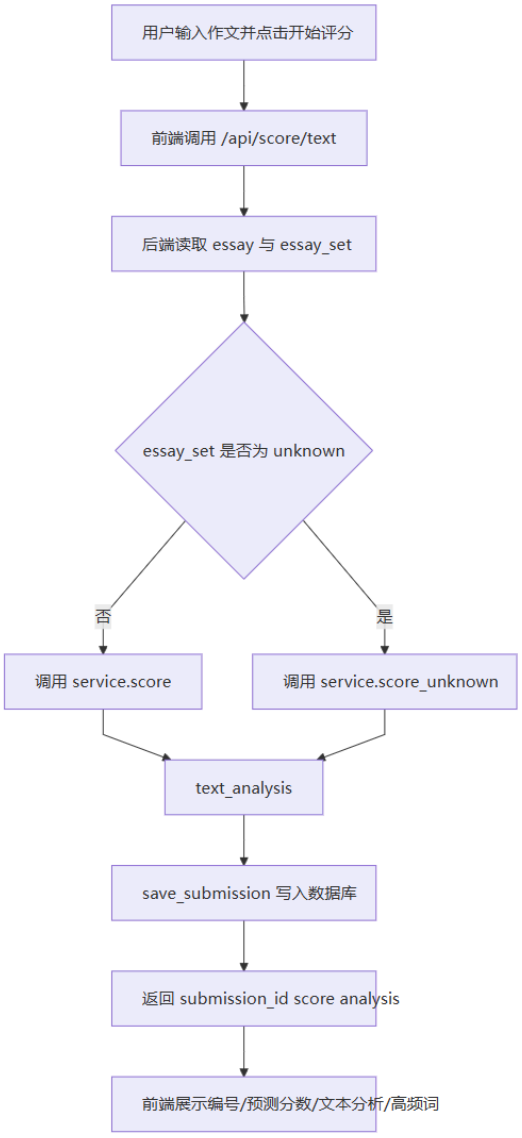
- 加载模型包与 tokenizer
- 对作文文本向量化
- 执行分数回归
- 反标准化恢复真实分数
- 对 `unknown` 执行多题目归一化融合

存储模块

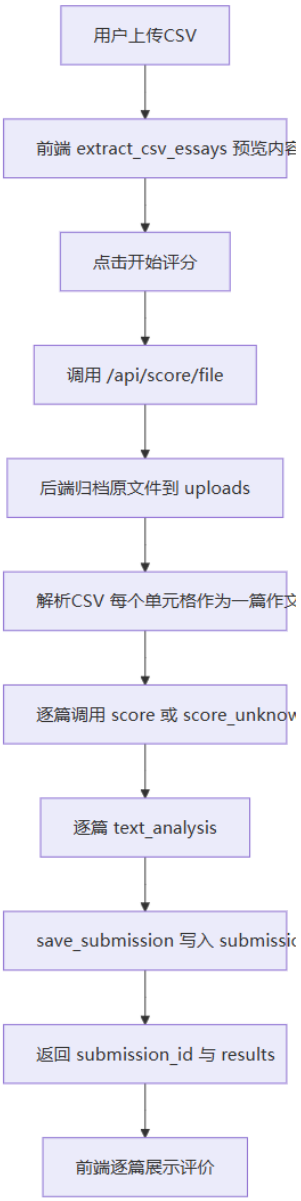
- 初始化 SQLite 表结构
- 保存 `submission` 与 `records`
- 提供分页查询、详情查询、删除能力

5. 执行视图（执行流程图）

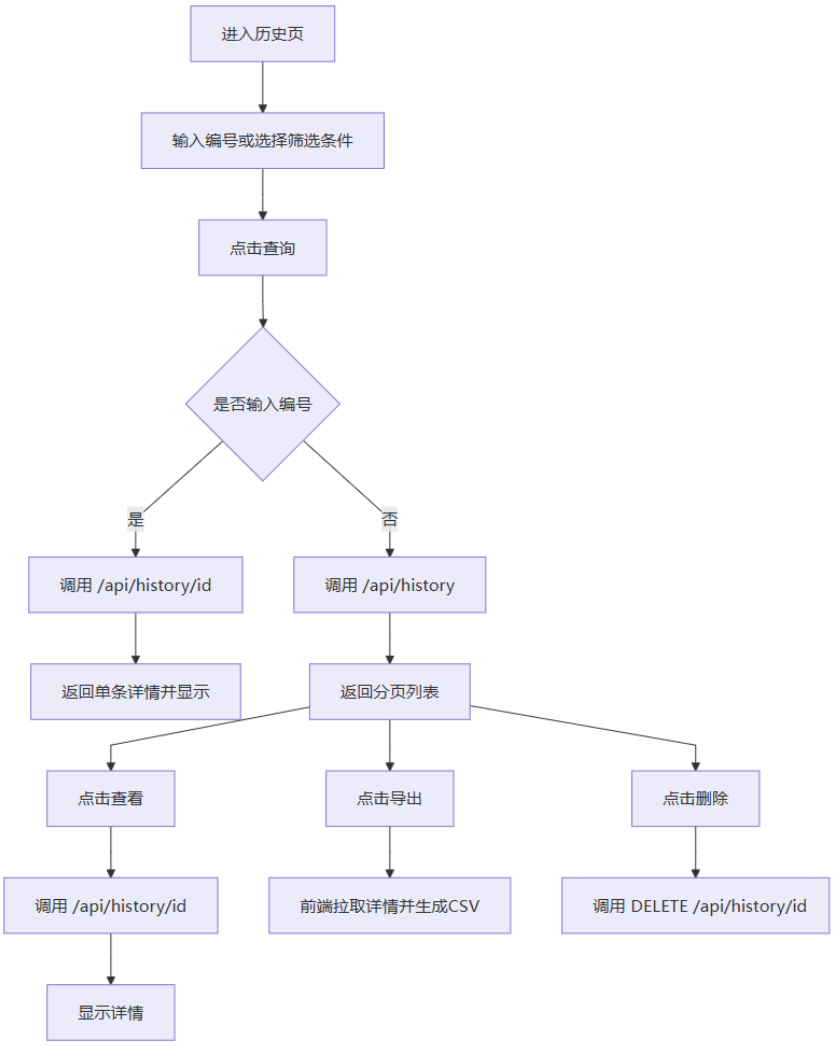
5.1 单篇评分执行流程


用户输入作文并点击开始评分
前端调用 /api/score/text
后端读取 essay 与 essay_set
essay_set 是否为 unknown
否
是
调用 service.score
调用 service.score_unknown
text_analysis
save_submission 写入数据库
返回 submission_id score analysis
前端展示编号/预测分数/文本分析/高频词

5.2 批量评分执行流程

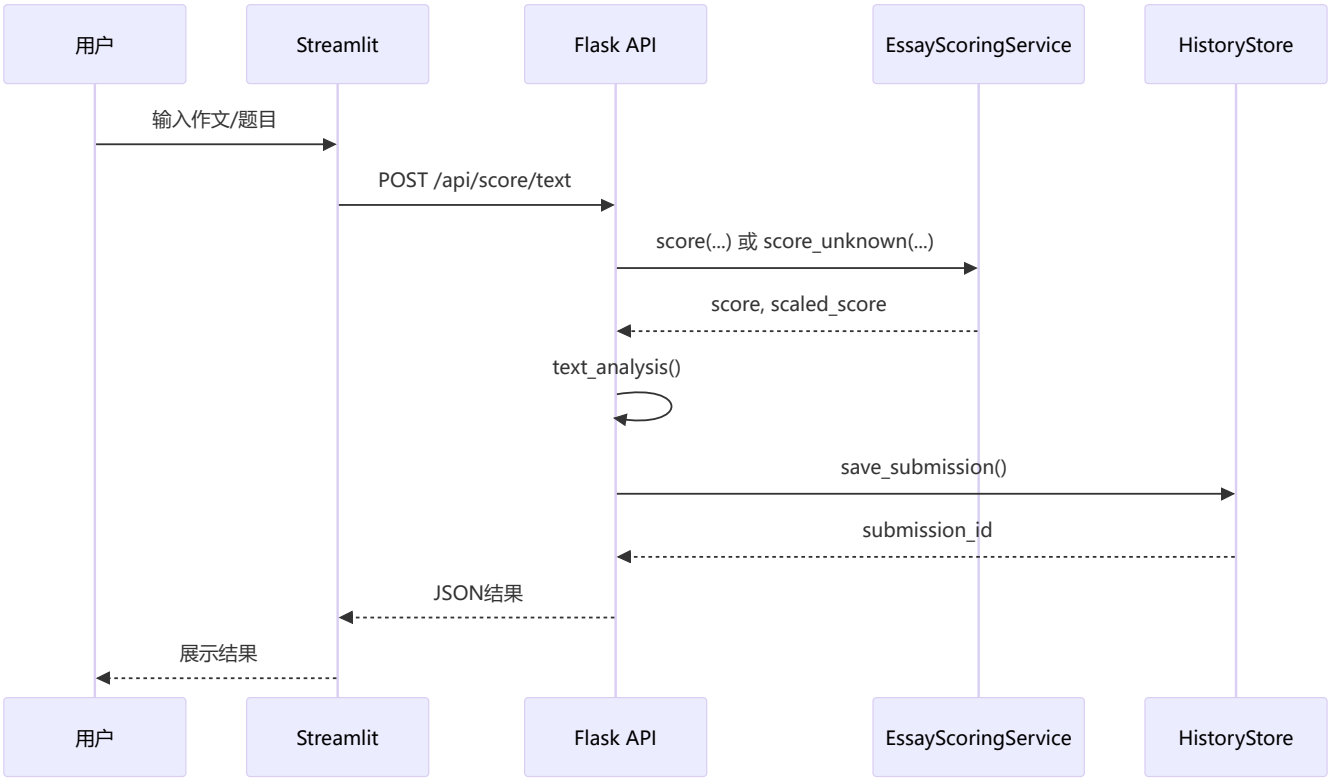

用户上传CSV
前端 extract_csv_essays 预览内容
点击开始评分
调用 /api/score/file
后端归档原文件到 uploads
解析CSV 每个单元格作为一篇作文
逐篇调用 score 或 score_unknow
逐篇 text_analysis
save_submission 写入 submissio
返回 submission_id 与 results
前端逐篇展示评价

5.3 历史记录执行流程

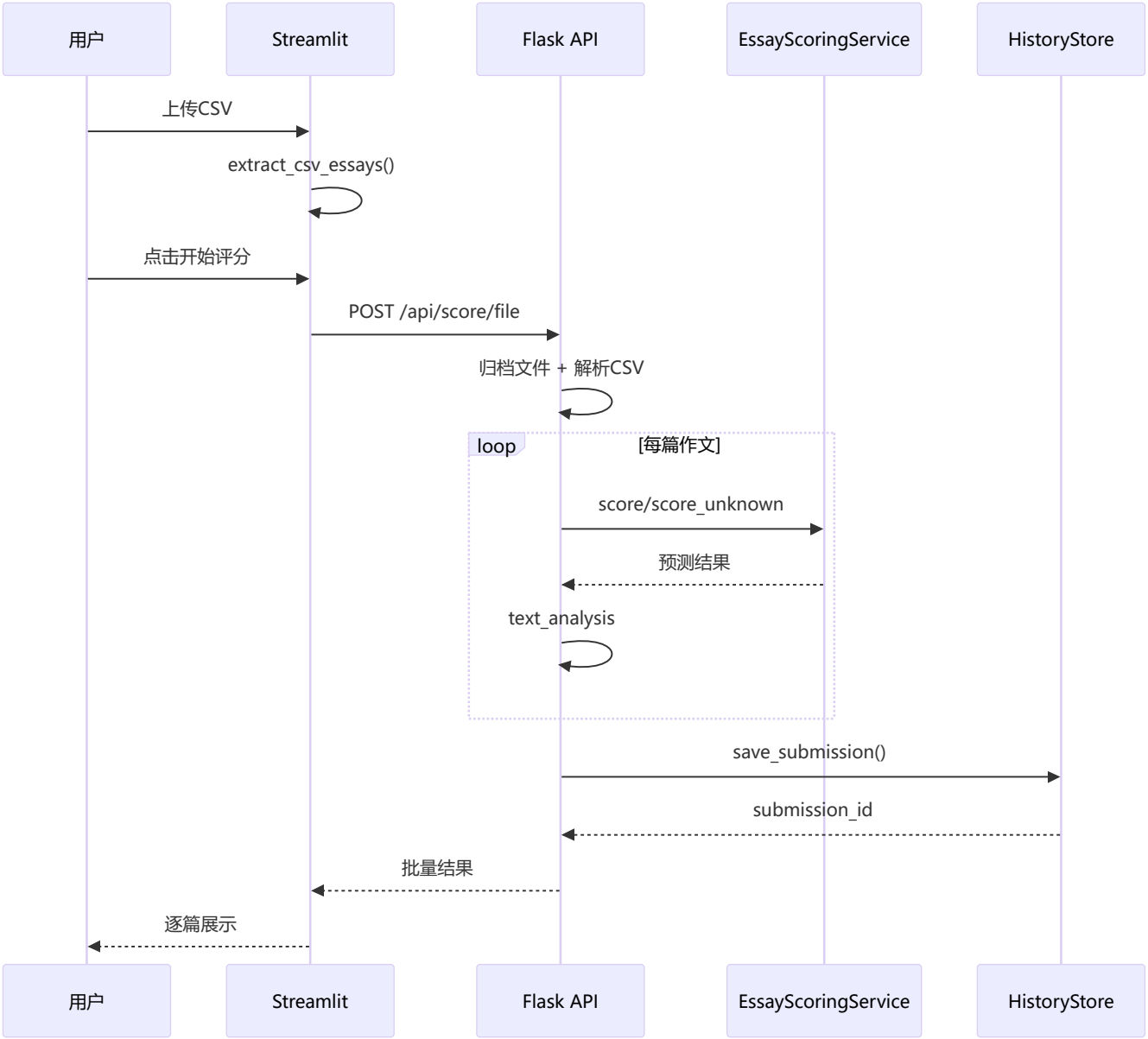


6. 动态建模

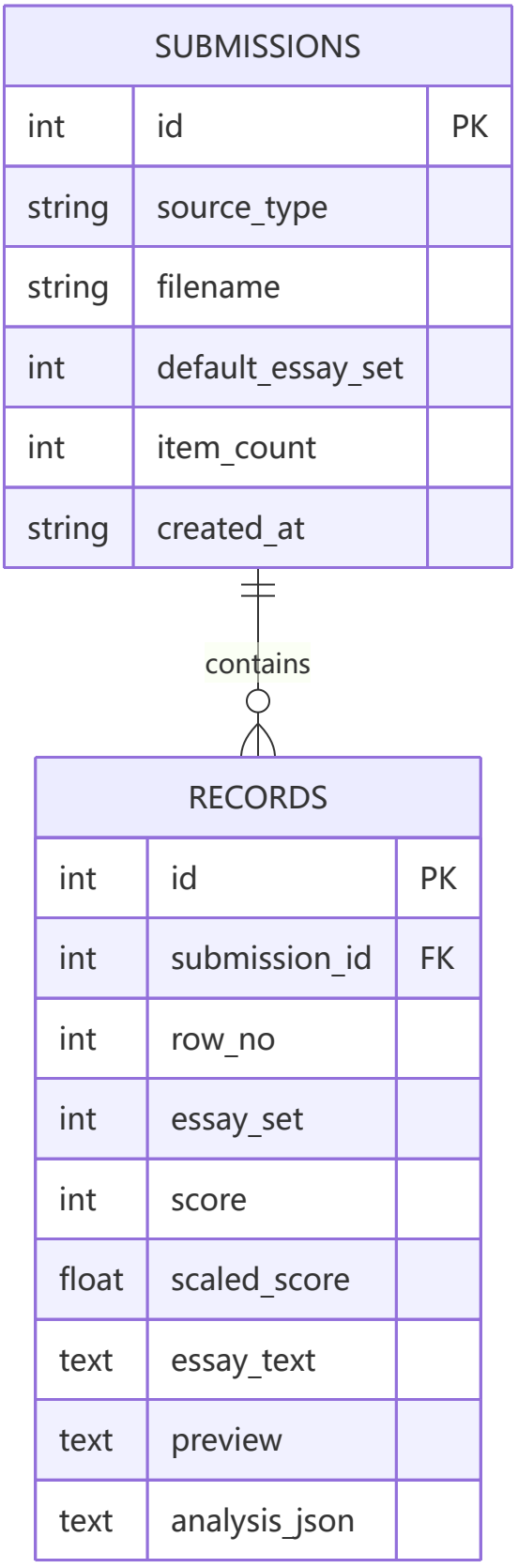
6.1 单篇评分时序图


用户
Streamlit
Flask API
EssayScoringService
HistoryStore
输入作文/题目
POST /api/score/text
score(...) 或 score_unknown(...)
score, scaled_score
text_analysis()
save_submission()
submission_id
JSON结果
展示结果
用户
Streamlit
Flask API
EssayScoringService
HistoryStore

6.2 批量评分时序图


用户
Streamlit
Flask API
EssayScoringService
HistoryStore
上传CSV
extract_csv_essays()
点击开始评分
POST /api/score/file
归档文件 + 解析CSV
loop
[每篇作文]
score/score_unknown
预测结果
text_analysis
save_submission()
submission_id
批量结果
逐篇展示
用户
Streamlit
Flask API
EssayScoringService
HistoryStore

6.3 数据模型图


SUBMISSIONS
int id PK
string source_type
string filename
int default_essay_set
int item_count
string created_at
contains
RECORDS
int id PK
int submission_id FK
int row_no
int essay_set
int score
float scaled_score
text essay_text
text preview
text analysis_json

7. 实现视图（源码结构）

7.1 目录结构

```
aeg/
├─ backend/
|   ├─ app.py
|   ├─ model_service.py
|   └─ storage.py
├─ frontend/
|   └─ streamlit_app.py
├─ data/
|   └─ history.db
├─ uploads/
├─ model/
├─ best_fold_model.pt
```

7.2 主要源码文件视图

7.3 主要类与函数

`backend/app.py`

- `text_analysis(text: str) -> dict`
  - 统计字符数、词数、句子数、段落数、平均句长、词汇丰富度、高频词
- `create_app() -> Flask`
  - 创建 Flask 应用并注册资源
- `HealthResource.get()`
  - 返回系统状态、模型路径、数据库路径、支持的题目集合
- `ScoreTextResource.post()`
  - 处理单篇评分请求
- `ScoreFileResource.post()`
  - 处理批量评分请求，支持文件归档与 CSV 解析
- `HistoryListResource.get()`
  - 历史分页列表与条件筛选
- `HistoryDetailResource.get()/delete()`
  - 历史详情与删除

`backend/model_service.py`

- `class MLP`
  - 回归头网络
- `class EssayScoreResult`
  - 标准题目评分结果结构体
- `class EssayScoringService`
  - `_load_package()`
  - `_build_score_ranges()`
  - `_build_roberta()`
  - `_build_tokenizer()`
  - `_build_mlp()`
  - `_embed_essays()`
  - `_predict_scaled()`
  - `_inverse_scale_score()`
  - `_normalize_score()`
  - `_fuse_unknown_score()`
  - `score()`
  - `score_unknown()`

`backend/storage.py`

- `class HistoryStore`
  - `_connect()`
  - `_init_db()`
  - `save_submission()`
  - `get_submissions()`
  - `count_submissions()`
  - `get_submission_detail()`
  - `delete_submission()`

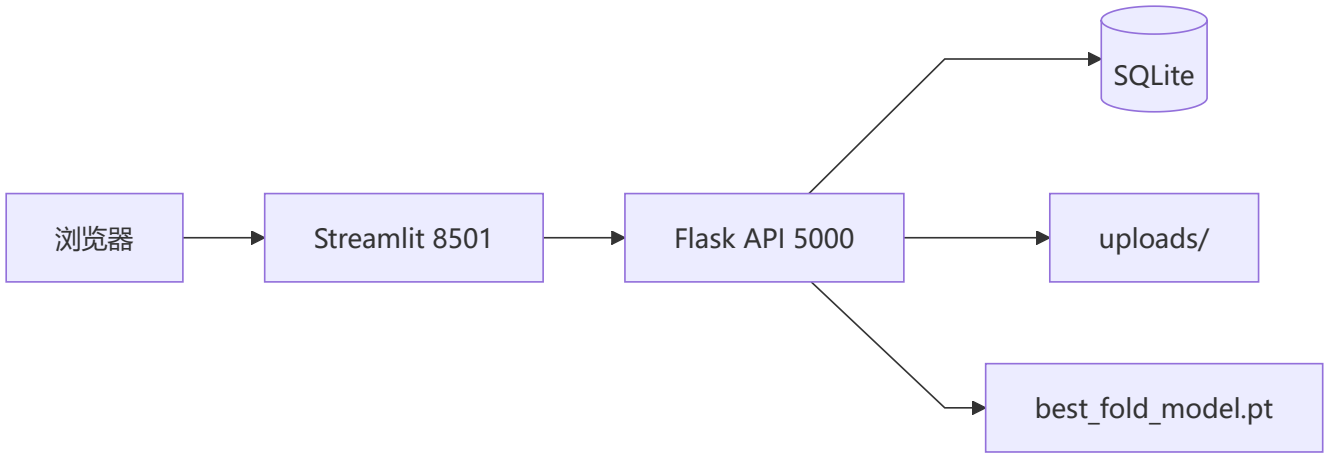
`frontend/streamlit_app.py`

- 接口调用函数
  - `call_health()`
  - `call_score_text()`
  - `call_score_file()`
  - `call_history()`
  - `call_history_detail()`

    - `call_history_delete()`
- 前端工具函数
    - `extract_csv_essays()`
    - `build_export_csv()`
    - `show_delete_confirm_dialog()`
    - `load_prompt_options()`
- 页面逻辑
    - 单篇评分页
    - 批量评分页
    - 历史记录页

8. 部署视图

8.1 本地部署视图



8.2 启动方式

后端:

```
python backend/app.py
```

前端:

```
streamlit run frontend/streamlit_app.py
```

9. 关键设计说明

9.1 `unknown` 评分策略

- 不直接输出某个题目集合的原分值
- 先对每个题目集合进行反标准化和归一化
- 再对所有集合的归一化分数取均值
- 最终输出 `0-100` 分

该策略适用于题目未知时的近似综合评分。

9.2 CSV 解析策略

- `header=None`
- 自动分隔符检测 `sep=None, engine="python"`
- 失败时回退到 `sep=","`
- 每个非空单元格都作为一篇作文

9.3 历史导出策略

- 导出行为由前端实现
- 先获取详情，再拼装 CSV
- 文件名使用提交编号
- 文本分析与高频词使用 JSON 字符串表示，避免丢失结构化信息

10. 风险与改进方向

10.1 当前风险

- 生产环境缺少鉴权与限流
- SQLite 更适合单机，不适合高并发
- 当前无自动化测试

10.2 改进方向

1. 增加单元测试与接口测试
2. 为历史记录增加按时间和分数区间筛选
3. 支持更多导出格式
4. 增加模型版本管理
5. 引入认证、日志与监控